Kafka и RabbitMQ

Семинар 6

Кирилл Дикалин

Институт прикладных компьютерных наук
Университет ИТМО

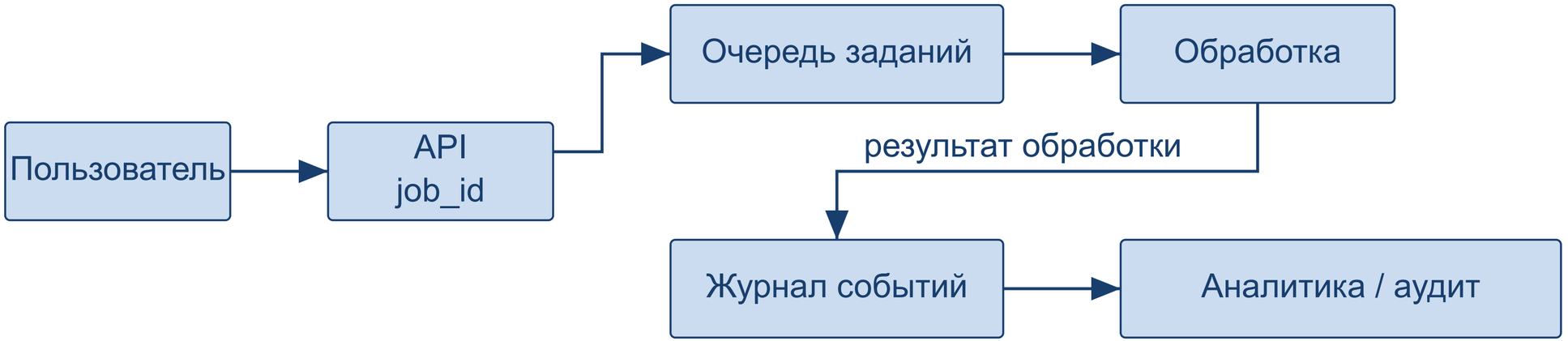
Откуда появились RabbitMQ и Kafka

Приложения работают с разной скоростью и могут быть временно недоступны. **Брокер принимает сообщения и передаёт их получателям**, которым необязательно быть готовыми одновременно с отправителем.

Система и происхождение	Исходная задача	Главное в устройстве
IBM MQSeries, 1993 Корпоративные системы IBM и других платформ	Связать разнородные приложения и надёжно передавать сообщения	Сохранение сообщений, очереди, восстановление после сбоев
RabbitMQ, 2007 LShift, Лондон, и CohesiveFT	Открытый брокер для обмена между приложениями по стандарту AMQP	Маршрутизация, очереди и подтверждение отдельных доставок
Kafka, 2011 Открытие исходников в LinkedIn	Собирать просмотры, поисковые запросы и другие события для оперативной и пакетной обработки	Масштабируемый журнал, последовательная запись и независимое чтение

Два разных требования. Передать работу исполнителю. Сохранить поток фактов для нескольких приложений.

Обработка изображения



Задание. Создать миниатюру изображения. Работу выполняет воркер.
Событие. Миниатюра готова. Факт нужен аналитике и аудиту.

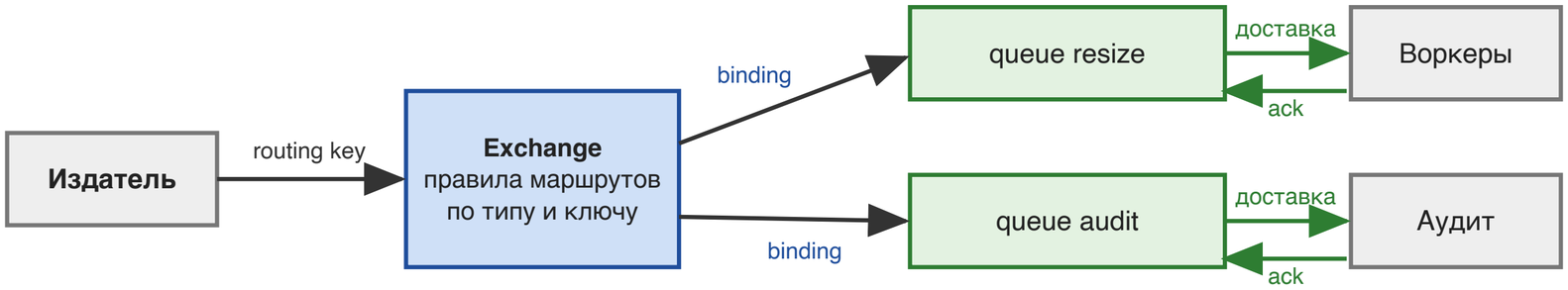
Ответ с `job_id` означает приём задания. Готовность результата — отдельное состояние.

Очередь задач и журнал событий

Вопрос	Очередь задач	Журнал событий
Что хранится?	Работа, ожидающая выполнения	Записи о произошедших событиях
Как делят работу?	Потребители одной очереди	Потребители одной группы
Что после обработки?	`ack` разрешает удалить сообщение	Сохраняется позиция группы
Можно перечитать?	После `ack` обычная очередь не хранит историю	Пока запись доступна по политике хранения
Независимые приложения	Отдельные очереди	Отдельные группы

Очередь сглаживает всплеск, но не увеличивает скорость обработки воркеров.

RabbitMQ. Exchange, binding, queue



сообщение копируется в каждую подходящую очередь и лежит там до подтверждения (ack)

- Издатель публикует в **exchange**. Его тип и **routing key** определяют, какие **bindings** подходят и в какие очереди попадут копии. Типы exchange — `direct`, `fanout`, `topic`, `headers`; безымянный exchange направляет по имени очереди.
- Несколько очередей — **несколько копий** (pub/sub). Несколько потребителей одной очереди — **разделение работы** (competing consumers). Одну доставку получает один из них.
- Издатель не знает потребителей. Маршруты можно менять без изменения его кода. **Брокер ведёт состояние доставки каждого сообщения**; `ack` идёт от потребителя к брокеру.
- Горячий путь отдельной реплики очереди ограничен одним ядром. Воркеры распараллеливают обработку; если предел достигнут в самой очереди, работу распределяют по нескольким очередям.

Маршрутизация и потребители

Тип exchange	Правило
`direct`	Точное совпадение routing key
`fanout`	Копия в каждую связанную очередь
`topic`	Шаблон. `*` — одно слово, `#` — ноль или больше
`headers`	Сопоставление заголовков сообщения

Безымянный exchange направляет по имени очереди.
`mandatory` позволяет обнаружить публикацию без маршрута.

Параллелизм воркеров и пропускная способность одной очереди — разные ограничения.

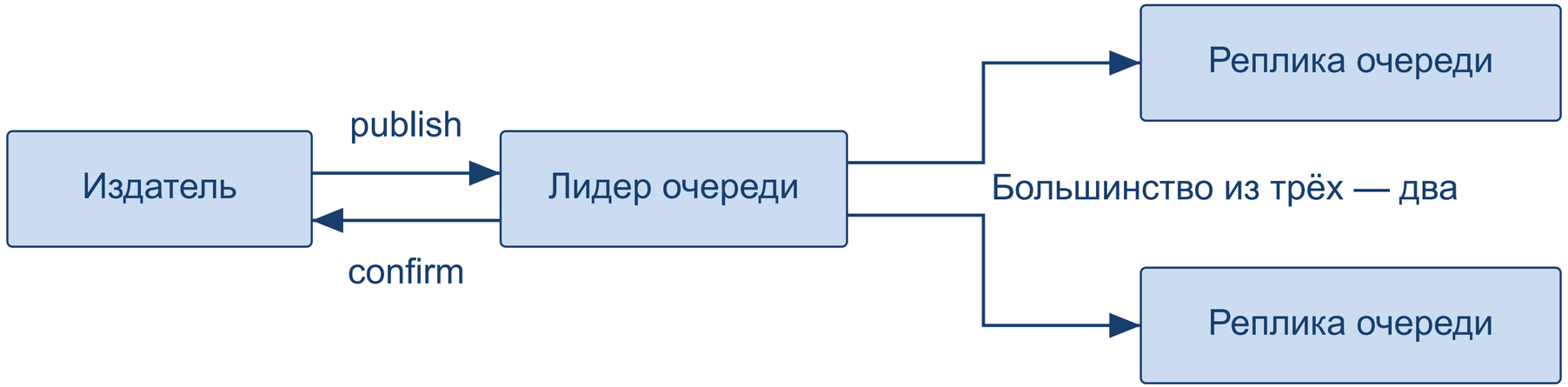
Три границы надёжности подтверждений

Участок	Механизм	Что подтверждено	Без него
Издатель → брокер	**Publisher confirms**	Persistent-сообщение в durable classic queue записано на диск; для quorum — принято кворумом реплик	Издатель не узнает о потере
Хранение	**Durable queue + persistent message** (`delivery_mode=2`)	Сообщение переживает рестарт с сохранным диском	Сообщение может исчезнуть при рестарте
Брокер → потребитель	**Manual ack**	Можно завершить доставку; при обрыве канала до ack сообщение возвращается	Auto-ack допускает потерю при сбое воркера

- При подтверждениях, повторной отправке и восстановлении получается доставка **хотя бы раз**. Нужен идемпотентный обработчик; `redelivered` помогает наблюдению, но не заменяет устойчивый `event_id`.
- Сброс persistent-сообщений на диск группируется. Подтверждение может задерживаться на сотни миллисекунд. Для потока используют **асинхронные confirms и пачки**, а не ожидание после каждого сообщения.
- AMQP-транзакции добавляют накладные расходы. Оценка снижения throughput в **250 раз** описывает приведённый в документации случай, а не любой сервер и нагрузку.

Confirm подтверждает приём брокером, а не завершение бизнес-операции.

Хранение и репликация очереди



Classic queue в RabbitMQ 4.2 не реплицируется.
Quorum queue хранит реплики и использует Raft. Для работы нужно большинство.

Durable не означает replicated. Три узла кластера не делают любую очередь трёхкратно реплицированной.

Prefetch и воркеры очереди задач

- `basic.qos`, **prefetch = p** — лимит неподтверждённых push-доставок. В обычном режиме RabbitMQ он действует **на потребителя**; при одном consumer на воркер у `m` воркеров в полёте не больше `p × m`. При заполнении окна новые доставки ждут `ack`.

Условия. Обработка t = 10 мс, доставка и ack RTT = 2 мс	Один последовательный воркер
`p = 1`. Между задачами ждём сеть	`1 / (t + RTT) ≈ 83 задач/с`
`p ≥ 2`. Следующая доставка может перекрыться с обработкой	Предел обработки `1 / t = 100 задач/с`

- Для **долгих и неравномерных задач** берут малое окно. Медленный воркер иначе удерживает задания. Для коротких 100–300 — ориентир для измерений, не универсальный оптимум. `prefetch=0` снимает лимит; `basic_get` им не ограничивается.
- Наблюдаем **ready, unacked, возраст старейшего задания, consumer capacity**. Больше воркеров увеличивает скорость обслуживания, пока ограничение находится у них.
- В RabbitMQ 4.2 тайм-аут consumer ack по умолчанию **30 минут**. Просрочка закрывает канал, неподтверждённые доставки возвращаются.
- **Порядок доставки не равен порядку завершения**. Несколько воркеров и redelivery меняют наблюдаемый порядок; single active consumer и последовательная обработка помогают сохранить его.

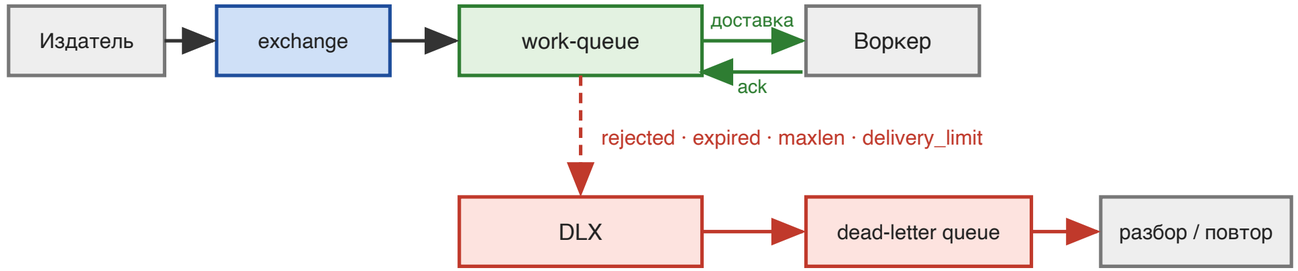
Повторная доставка и результат

```
1. Воркер получил сообщение
2. Сохранил результат в БД
3. Упал до отправки ack
4. Брокер доставил сообщение повторно
```

Одна бизнес-операция может получить несколько доставок.

Защиту обеспечивают стабильный `event_id` и одна транзакция БД.
Она объединяет `проверку / запись event_id + изменение результата`.

Dead Letter Queue и отравленные сообщения



заголовок x-death хранит причину, очередь и счётчик; без DLX такие сообщения теряются

- Четыре причины — **rejected** (`reject` / `nack`, `requeue=false`), **expired** (TTL), **maxlen** (вытеснение при лимите очереди), **delivery_limit** (лимит повторных доставок quorum queue). `x-death` хранит причину, очередь и счётчик.
- **Отравленное сообщение** падает при каждой обработке. У quorum queues RabbitMQ 4.2 лимит по умолчанию **20**; без настроенного DLX после исчерпания лимита сообщение отбрасывается.
- **Отложенный повтор.** Отдельная retry-очередь без потребителей, TTL и обратный DLX в рабочий маршрут. Нужны предел числа попыток и задержка; бесконечный цикл не исправляет данные.
- У DLQ должны быть **владелец и алерт**. Quorum queues поддерживают настраиваемый `at-least-once` dead-lettering; обычный режим — `at-most-once`. Наличие DLQ само по себе не гарантирует сохранность переноса.

Демо A. Маршрут, сбой и повтор

1. Две очереди и два воркера. Кто получит копию?
2. Сбой после результата, до `ack`. Что повторится?
3. Проверка `event_id` в транзакции. Изменится ли счётчик ещё раз?
4. Некорректное изображение. Где окажется сообщение?

```
./lab.sh rabbit routing
./lab.sh rabbit redelivery
./lab.sh rabbit idempotency
./lab.sh rabbit dlq
```

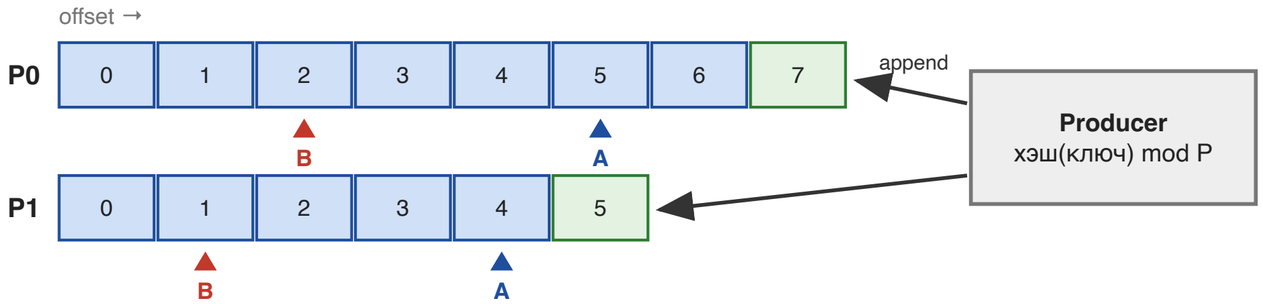
Проверяем получателей, `redelivered`, счётчик результатов, причину в DLQ.

Что подтверждает RabbitMQ

Наблюдение	Вывод
6 заданий у воркеров, 6 записей аудита	Разделение внутри очереди, копирование между очередями
Воркер упал до `ack`, пришёл `redelivered`	Незавершённая доставка вернулась
Счётчик без защиты = 2, с защитой = 1	Повтор доставки отделён от повторного эффекта
В DLQ тот же `event_id`	Ошибка обработки сохранена для разбора

Подтверждение доставки не является транзакцией между брокером и вашей БД.

Kafka. Topic, partition, offset



A и B — две независимые группы потребителей, у каждой свой offset; чтение ничего не удаляет

- **Topic** — именованный поток записей, разделённый на **partitions**. Каждый раздел — **журнал только для дописывания**; **offset** обозначает позицию записи внутри него. Записи P0/offset 0 и P1/offset 0 различаются.
- Порядок задан **только внутри раздела**. На схеме ключ выбирает раздел через `hash(key) mod P`; одинаковый ключ сохраняет локальность при неизменном числе разделов и алгоритме. Клиент может явно выбрать раздел.
- **Чтение не удаляет записи.** Группы A и B хранят независимые offsets, могут отставать и читать заново в пределах сохранённой истории. Модель ближе к журналу БД, чем к очереди удаляемых заданий.
- Раздел — единица **порядка, параллелизма, репликации и хранения**. Bootstrap-сервер сообщает метаданные, затем producer обращается к лидеру нужного раздела.

Почему Kafka быстро обрабатывает поток

Приём	Механизм	Эффект и ограничение
Дописывание файла сегмента	Последовательный ввод-вывод	Меньше случайных обращений; выигрыш зависит от носителя и нагрузки
Page cache ОС, а не отдельный кэш данных в heap JVM	Read-ahead и write-behind	Меньше двойного кэширования и работы GC; кэш может сохраниться после рестарта процесса
Пачки с настройками `batch.size`, `linger.ms`; сжатие	Амортизация фиксированной цены	Меньше мелких операций диска и сети; ожидание пачки добавляет задержку
`sendfile` при подходящем пути передачи	Из файлового кэша в сокет средствами ОС	Нет промежуточного буфера приложения; TLS и преобразования могут менять путь
Разрежённый индекс offset → позиция	Поиск близкой позиции и последовательное дочитывание	Компактный индекс; это не гарантия O(1) для любой операции

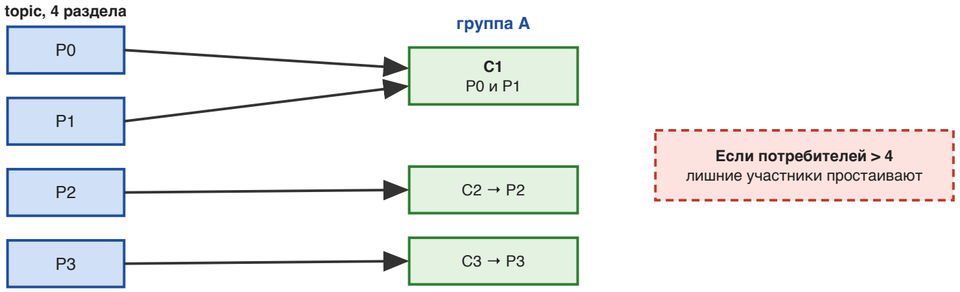
- Kafka оптимизирует **дописывание и последовательное чтение**. Поиск произвольного поля и обновление записи на месте не входят в эту модель журнала.
- Исторический пример на 6 HDD. Около **600 МБ/с** линейного I/O против **100 кБ/с** случайных записей — примерно **6000×**. Это иллюстрация устройства диска, не обещание производительности Kafka или SSD.
- Выгодно читать **подряд**. Отставший потребитель может читать с диска вместо page cache; конкурирующие читатели расходуют общую пропускную способность.

Retention, compaction и повторное чтение



- Чтение не удаляет данные. При `cleanup.policy=delete` действуют **retention по сроку и/или объёму**. Типичные умолчания Kafka 4.1 — 7 суток, лимит объёма не задан, `segment.bytes=1 GiB`. Удаляются **закрытые сегменты**, место освобождается порциями; отстающий читатель их не удерживает.
- **Replay.** Сдвинуть позицию назад для пересчёта агрегата, нового потребителя, восстановления кэша или индекса. Повторятся и внешние эффекты, если от них нет защиты.
- `cleanup.policy=compact` сохраняет последнее значение ключа **по мере фоновой очистки**. Несколько версий могут сосуществовать; удаление ключа — tombstone с **null**, а не пустой строкой. Политики `compact,delete` можно сочетать.
- **Tiered storage.** При настройке старые сегменты могут храниться удалённо, на брокере остаётся горячая часть. Нужна соответствующая реализация удалённого хранения.
- Ёмкость ≈ **байт/с × срок хранения × RF**. 20 МБ/с × 604 800 с × 3 ≈ **36 ТБ**; учесть сжатие, индексы, служебные данные и запас.

Consumer group



- В обычной группе **раздел назначен одному потребителю**, но потребитель может читать несколько. **P0 и P1 → C1**, P2 → C2, P3 → C3. При числе участников больше числа разделов лишние простаивают. Другая группа читает независимо.
- **Committed offset** хранится в `__consumer_offsets`. Lag ≈ конец доступного журнала − committed offset; оценка задержки **W ≈ lag / μ** требует сопоставимых записей и устойчивой скорости обработки μ.
- **Оценка размера.** 20 000 сообщений/с × 5 мс = 100 одновременно занятых последовательных обработчиков. При целевой загрузке 70% — **⌈100 / 0,7⌉ = 143** обработчика и не меньше разделов для такого параллелизма, если нагрузка равномерна.
- Разделы выбирают заранее. Уменьшить число разделов существующего топика нельзя; увеличение меняет `hash(key) mod P` для будущих записей. Старые записи не перемещаются, горячий ключ остаётся ограничением.
- **Ребалансировка** меняет назначения при входе и выходе участников. Новый протокол `consumer` (KIP-848) переносит назначения на брокер и перераспределяет инкрементально; в демо используется `classic`.

Обработка и commit offset

После обработки offset `7` сохраняем `8` — **следующую позицию**.

Момент commit	Семантика при восстановлении
До бизнес-эффекта	Не более одного раза. Работа может быть пропущена
После бизнес-эффекта	Хотя бы раз. Работа может выполниться повторно

`position` — текущая позиция клиента.
`committed offset` — сохранённая позиция группы для восстановления.

Commit offset не удаляет запись и не подтверждает транзакцию внешней БД.

Демо B. Группы, replay и commit

1. Два раздела, три участника группы. Кому достанется работа?
2. Вторая группа. Сможет ли прочитать те же события?
3. Выполняем `seek(0)`. Останутся ли прочитанные записи?
4. Сбой по разные стороны commit. Пропуск или повтор эффекта?

```
./lab.sh kafka groups
./lab.sh kafka replay
./lab.sh kafka commit
```

Проверяем назначения разделов, offset, сохранённый прогресс, результат обработки.

Репликация, ISR и подтверждения



acks	Когда producer считает отправку завершённой	Риск при отказе лидера
`0`	Не ждёт ответа брокера	Не узнает, принял ли брокер запись
`1`	Лидер записал в локальный журнал	Может исчезнуть хвост, не полученный другими репликами
`all`	Записали все текущие ISR; соблюдён `min.insync.replicas`	Подтверждённая запись сохраняется, пока остаётся живая синхронизированная копия

RF=3, ISR={1,2}. Третья реплика отстала. **High watermark=10**. Offsets < 10 доступны обычному чтению; `read_committed` дополнительно ограничен незавершёнными транзакциями.

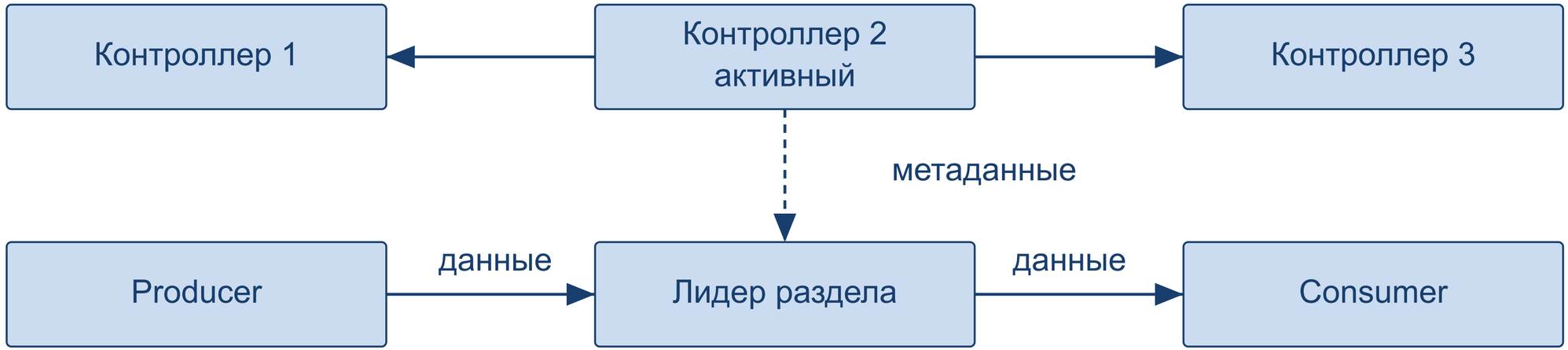
Подтверждение не означает fsync на каждой реплике. В обычном режиме запись уже в page cache всех ISR; отдельный fsync для каждой публикации не требуется. Отказ одного узла покрывают другие копии. Одновременная потеря всех реплик и их несброшенного кэша может потерять хвост. Нужны разные машины, стойки и зоны.

acks и минимальный ISR

Настройка	Что означает
`acks=all`	Подтверждение всех текущих ISR; у Java producer Kafka 4.1 это значение по умолчанию
`min.insync.replicas=1`	Стандартное значение без переопределения; ISR может состоять из одного лидера
`replication.factor=3`, `min.insync.replicas=2`	Типичная конфигурация для продолжения записи при потере одной реплики
`unclean.leader.election.enable=false`	Запрет выбора несинхронизированной реплики ценой возможной недоступности

- При **ISR=1 и minISR=1** `acks=all` ждёт только лидера. Слово `all` само по себе не создаёт вторую копию.
- При **RF=3, ISR=2, minISR=2** запись с `acks=all` возможна. При **RF=3, ISR=2, minISR=3** она отклоняется, хотя лидер и чтение могут оставаться доступными.
- `minISR=2` не значит «дождаться любых двух», если в текущем ISR три. При `acks=all` участвует **текущий ISR**, а порог ограничивает его минимальный размер.
- Ошибка или таймаут не всегда доказывают отсутствие записи. Подтверждение могло потеряться. Повторы и устойчивый идентификатор события проектируют отдельно.

Контроллеры KRaft



Контроллеры согласуют **метаданные** — состав кластера и назначения разделов.
Брокеры обслуживают **данные** — публикацию, чтение и репликацию разделов.

Большинство контроллеров и ISR раздела — разные условия.
Совмещение ролей удобно локально; их назначение от этого не меняется.

Демо C. Отказ узла и запись

1. Зафиксировать лидера, RF=3 и ISR=3.
2. Остановить лидера. Получить нового, прочитать прежнюю запись и записать новую.
3. При ISR=2 поднять minISR до 3. Проверить отказ записи.
4. Вернуть minISR=2 и восстановить узел.

```
./lab.sh kafka baseline
./lab.sh kafka failover
./lab.sh kafka min-isr
```

Проверяем лидера, ISR, подтверждение записи и конкретную ошибку брокера.

Гарантии доставки и подводные камни

Семантика	Как получить	Цена
Не более одного раза	Commit offset **до** обработки	При сбое работа пропускается
Хотя бы раз	Commit offset **после** обработки	Возможны дубли; нужен идемпотентный обработчик
Ровно один раз внутри Kafka	Идемпотентный producer; выходные записи и входные offsets в одной транзакции; `read_committed`	Накладные расходы и задержка; внешние БД и HTTP вне этой транзакции

- **Идемпотентный producer** убирает повторы отправки, не две новые бизнес-публикации. У Java producer 4.1 включён по умолчанию при совместимых настройках; у других клиентов проверить конфигурацию.
- **Отравленная запись** блокирует раздел при политике «не пропускать». В обычной группе повторы, топик ошибок и commit согласует приложение. У **share groups Kafka 4.4** появилась отдельная серверная DLQ; это другой режим, отсутствующий в демо 4.1.
- **Медленный потребитель** задерживает свои назначения; другие потребители могут продолжать. **Перекос ключей** создаёт горячий раздел.
- **Избыток разделов** расходует файловые дескрипторы, память и mmap; увеличивает цену восстановления и ребалансировок. Проверяют и лимит `vm.max_map_count`.

Границы exactly-once

Механизм	Где действует гарантия
Идемпотентный producer	Убирает дубли определённых повторов отправки в Kafka
Транзакция Kafka + `read_committed`	Выходные записи и входные offsets фиксируются вместе
`event_id` + транзакция БД	Защита конкретного бизнес-эффекта в этой БД

HTTP-вызов, письмо или запись в отдельную БД автоматически в транзакцию Kafka не входят.

Два новых `produce()` с одинаковым JSON могут создать две записи.

Что происходит с данными в четырёх моделях

Вопрос	БД	Кэш Redis	Очередь RabbitMQ	Журнал Kafka
Роль	**Источник истины**	**Восстановимая копия**	**Работа**	**Факты за срок хранения**
После чтения	Данные остаются	Копия остаётся	`ack` разрешает удалить	Запись остаётся; у группы свой offset
Перечитать	Да	Пока не вытеснено	После ack — нет	В пределах retention
Порядок	Транзакции, ключи	Зависит от операции	Доставка и завершение различаются	Внутри раздела
Единица масштаба	Секция, шард	Слот, шард	Очередь, воркеры	Раздел
Что ломается	Дорогая запись и индексы	Потеря, вытеснение, stampede	Рост очереди, отравленные сообщения	Горячий ключ, lag, retention
Наблюдение	p99, lag реплик	Hit ratio, память	Ready, unacked, **возраст**	Consumer lag, ISR

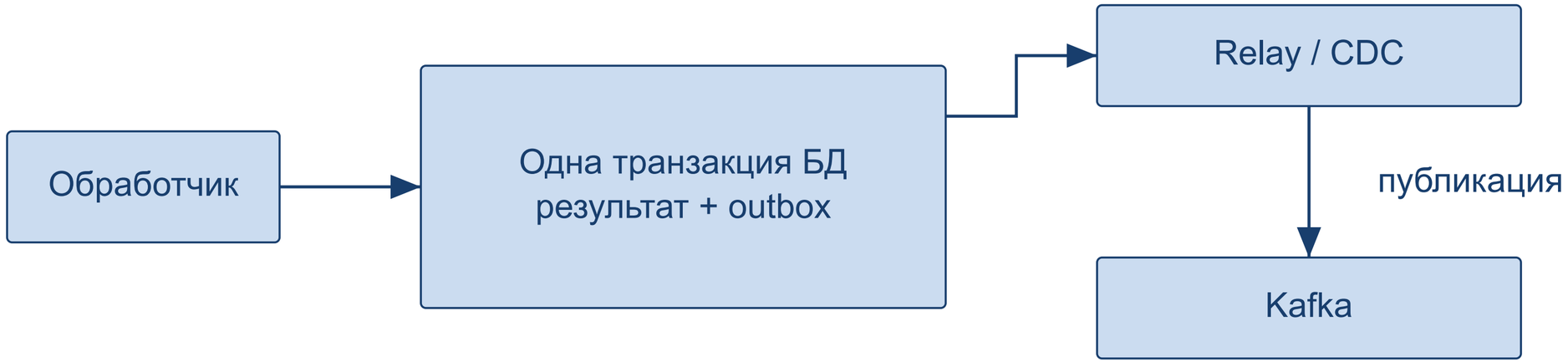
Очередь хранит **работу**. После подтверждения она исчезает. Журнал хранит **факты**. Сколько раз читать доступную историю, решает приложение.

Выбор по характеру нагрузки

Нагрузка или требование	Модель и инструмент	Цена
Поиск по ключу и условию, транзакции, долговечность	Хранилище. PostgreSQL и аналоги	Запись и обслуживание индексов
Повторные чтения, источник насыщен, данные восстановимы	Кэш. Redis, cache-aside	Устаревание, stampede, холодный старт
Тяжёлая операция вынесена из запроса; задачу делает один воркер	Очередь задач. RabbitMQ	Ack, идемпотентность, DLQ
Факт нужен нескольким приложениям, нужен replay, большой поток	Журнал событий. Kafka	Порядок в разделе, lag, retention
Работа с отдельными подтверждениями или повторное чтение в другом продукте	Kafka share groups / RabbitMQ Streams	Другие гарантии порядка; проверить возможности версии
Нагрузка мала, СУБД уже есть	Таблица заданий. `SELECT … FOR UPDATE SKIP LOCKED`	Конкуренция за ресурсы БД, собственный протокол выполнения

Новый компонент должен решать измеренную проблему. Для каждой строки нужны объём, задержка, допустимые потери и порядок — название продукта само по себе их не определяет.

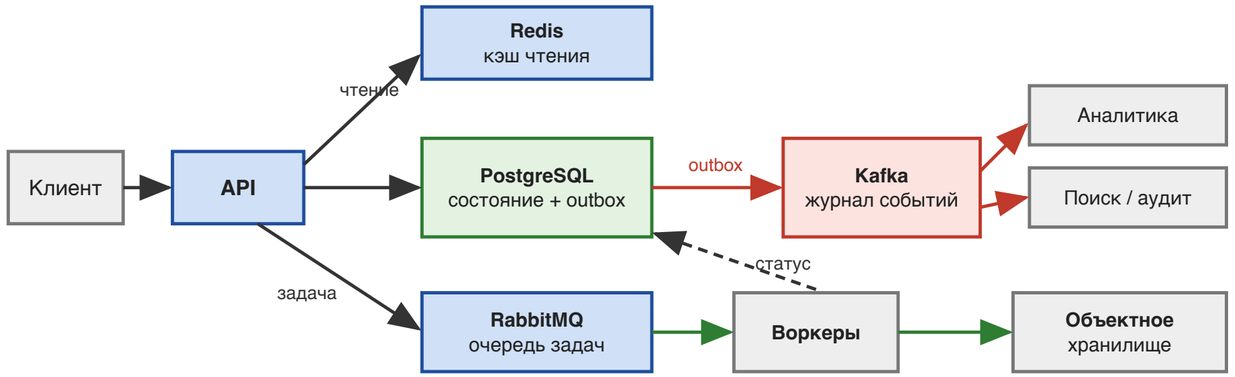
Состояние в БД и публикация события



Проблема. Результат сохранён, процесс упал до публикации события.

Outbox. Результат и запись о событии фиксируются одной транзакцией.
Отдельный relay публикует событие. Повтор relay возможен — `event_id` сохраняется.

Как сочетают модели



- **PostgreSQL — источник истины**, Redis ускоряет чтение. **API → RabbitMQ → воркеры → результат**. Клиент получает идентификатор, состояние задания хранится в БД, файлы — в объектном хранилище.
- **Kafka хранит факты** для аналитики, поиска и аудита. Результат и outbox фиксируются одной транзакцией БД; relay переносит события в Kafka. Стрелка «outbox» обозначает этот процесс, а не самостоятельную отправку таблицей.
- **API → RabbitMQ** — прямая публикация задания. Если API отдельно меняет БД и публикует сообщение, между действиями есть окно сбоя. Outbox для задания связывает намерение отправки с транзакцией БД; его нужно спроектировать отдельно.
- **Границы моделей размываются.** RabbitMQ Streams дают журнал и replay; Kafka share groups допускают отдельные подтверждения записей и иную модель распределения. Возможности и зрелость зависят от версии; в демо — обычные consumer groups.

Ёмкость и наблюдаемость

При входе **500 задач/с** и обработке **400 задач/с**
очередь растёт на **100 задач/с**, за 60 с — на **6000 задач**.

При обработке 800 задач/с и том же входе
накопленное убывает на **300 задач/с** → примерно **20 с** до опустошения.

RabbitMQ	Kafka
Ready, unacked, возраст работы, redelivery, DLQ	Lag по разделам, возраст событий, ISR, ошибки producer

Общий средний lag может скрыть один перегруженный раздел.

Чек-лист инженера

1. **«OK» на каждой границе.** Что подтверждено, где сохранено, какое окно потери допустимо? Из этого следуют `fsync`, репликация, `acks` и confirms.
2. **Диск и очередь.** IOPS × размер операции; `L = λ × W`; group commit и пачки. При ограничении диска проверить `iostat -x`, при очереди — вход и скорость обслуживания.
3. **Индексы.** B+-дерево и LSM выбирают по смеси чтений и записей. Измеряют p99, усиление записи и всплески compaction.
4. **Redis.** Можно ли восстановить данные? Запас памяти для `fork`, политика `maxmemory`, горячие и большие ключи; избегать блокирующего `KEYS` в рабочей нагрузке.
5. **RabbitMQ.** Confirms + manual ack + идемпотентность; prefetch по RTT и длительности задачи; DLX, лимит повторов и возраст старейшего задания.
6. **Kafka.** Ключ и порядок; для последовательных обработчиков `P ≥ ⌈λ × t / ϱ⌉`; `acks=all`, `RF=3`, `minISR=2`; lag, ISR и размещение реплик.
7. **Цена нового компонента.** Измеримое ограничение, критерий улучшения, наблюдение и способ отката. Компонент требует эксплуатации и создаёт свои режимы отказа.

Разбор решения

Задача. Создать миниатюры, уведомить аудит и аналитику, разрешить пересчёт аналитики.

1. Где хранится задание, а где факт его выполнения?
2. Воркер упал после записи в БД. Что защитит результат от повторного эффекта?
3. У группы два раздела и три участника. Увеличится ли параллелизм чтения?
4. RF=3, ISR=2, minISR=3. Должна ли запись с `acks=all` пройти?

Самостоятельно. Изменить одно условие эксперимента, записать прогноз и объяснить результат.

Заключение

Тема	Главное
Хранение	Случайный доступ к HDD — порядок 10 мс; последовательный I/O дешевле. Исторический пример 600 МБ/с против 100 кБ/с даёт 6000×, не универсальную гарантию. Пропускная способность ≈ IOPS × размер; $L = \lambda \times W$
`fsync` и WAL	`write` — в кэш ОС; `fsync` — запрос устойчивого сброса. **WAL** — журнал до записи данных; group commit делит цену fsync на b коммитов
B+-дерево	Оценка высоты $h \approx \lceil \log_f N \rceil$. Около 4 при $N=10^9$, $f=500$. Изменение записи загрязняет страницу; её запись дороже изменения нескольких байтов
LSM-дерево	Memtable → SSTable → compaction; усиление записи при leveled compaction грубо растёт как $T \times$ число уровней; фильтры Блума сокращают лишние чтения
Redis	Выполнение обычных команд последовательно; **RDB** — снимки, **AOF** — журнал операций; асинхронная репликация; горячие и большие ключи
RabbitMQ	Exchange → binding → queue; confirms + manual ack + повторы дают **хотя бы раз**; идемпотентность, prefetch, **DLX**
Kafka	Раздел — журнал; offset, группы; `acks=all` + **ISR** + minISR; retention, compaction и replay
Выбор	Кэш хранит **копию**, очередь — **работу**, журнал — **факты**

Обозначения. IOPS — операций ввода-вывода/с; L — среднее число запросов в системе; λ — вход/с; W — среднее пребывание; b — коммитов в пачке; N — число ключей; f — ветвление; T — отношение размеров уровней LSM; RF — число реплик; ISR — синхронизированные реплики; DLX — exchange для недоставленных/отклонённых сообщений.

RabbitMQ и подтверждения

- Семён Мартынов и соавторы. L6, схемы и архитектура брокеров
- RabbitMQ. Объявление первого выпуска, 2007
- IBM MQSeries. История и задачи обмена сообщениями
- AMQP, очереди и маршрутизация
- Publisher confirms и consumer acknowledgements
- Quorum queues
- Consumer prefetch
- Dead letter exchanges

Kafka и обработка событий

- LinkedIn. Открытие исходников Kafka, 2011
- Kreps, Narkhede, Rao. Kafka, NetDB 2011
- Архитектура Kafka
- KRaft
- Конфигурация топиков и min.insync.replicas
- Kafka 4.4. DLQ для share groups
- Transactional outbox. Debezium